SELF-STUDY CURRICULUM

Agent Engineering

Project 1 · Build a ChatGPT/Claude-Class Assistant and Deep Research Agent

VERSION	v3.0 — merged edition
DATE	25 August 2026
AUDIENCE	ML + LLM fundamentals, basic ADK
EFFORT	~253 hours · 8–10 months @ 10 hrs/wk
PHASES	18, across rounds R0 → R3
OUTCOME	A harness you built, measured, broke, fixed, and can rebuild without a framework

Contents

Appendix D — The habit this project should teach

How to read this document

This is a **learn-by-building curriculum**, not a reading list. Every phase follows the same loop:

```
READ THE CONCEPT
       │
       ▼
BUILD THE MINIMUM VERSION
       │
       ▼
INSTRUMENT IT                    ◄── you must be able to see what happened
       │
       ▼
WRITE EVALS
       │
       ▼
BREAK IT INTENTIONALLY           ◄── the failure is the lesson
       │
       ▼
IMPROVE IT
       │
       ▼
WRITE DOWN WHAT YOU LEARNED
```

A phase is **not** complete because the feature worked once. A phase is complete when all six are true:

```
[ ] 1.  You can explain the underlying problem without framework vocabulary
[ ] 2.  You have implemented the feature
[ ] 3.  You can inspect what the agent did
[ ] 4.  You can measure whether it works
[ ] 5.  You know at least one important failure mode
[ ] 6.  You have a regression test for that failure
```

Notation used throughout

MARKER	MEANING
`RULE`	A design constraint. Violating it costs you weeks later.
`TRAP`	A specific, common failure mode. Named so you recognise it.
`VERIFIED`	Checked against a primary source on 25 Aug 2026. See Appendix C.
`VENDOR`	Self-reported by the vendor. Directional only — reproduce it yourself.
`EXIT`	The demonstrable exit criterion for a phase.

PART I — FOUNDATIONS

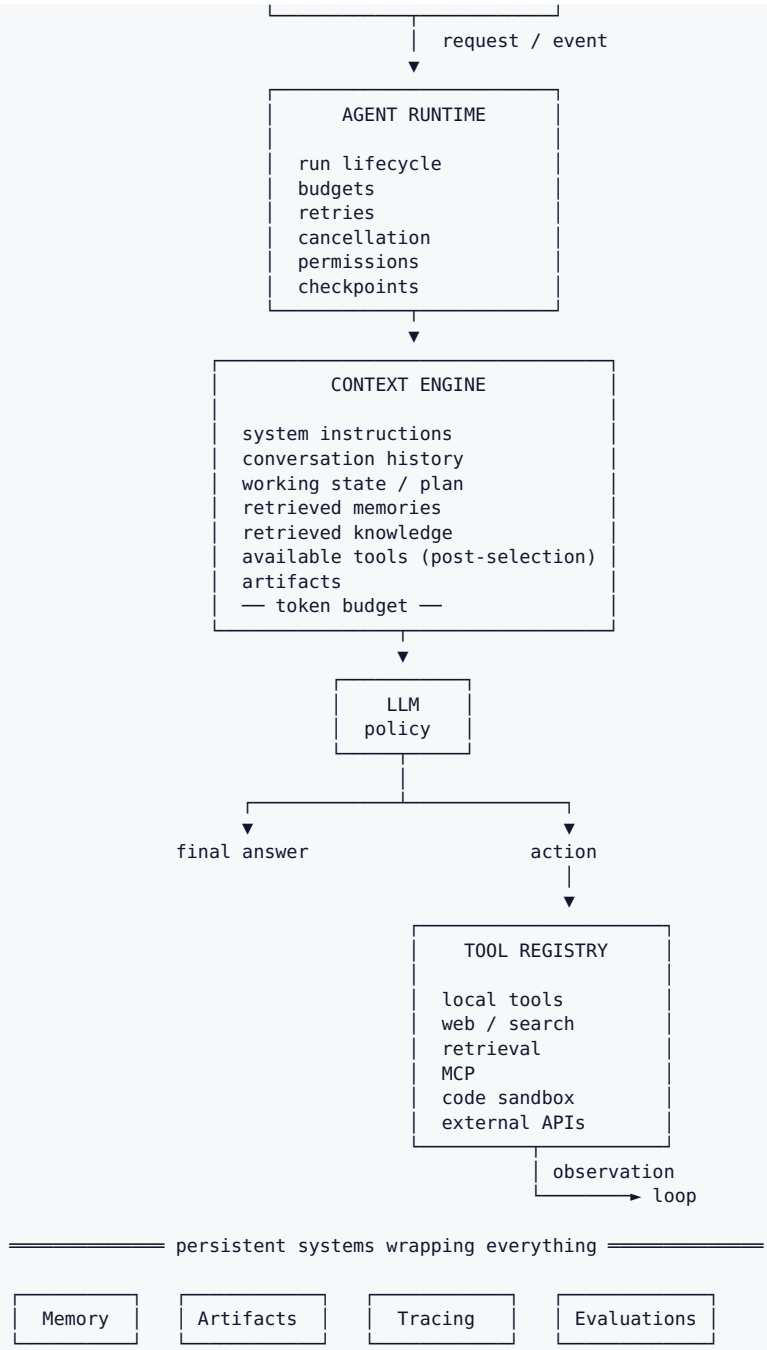
Read Part I once, end to end, before writing any code. It is about four hours and it determines whether Phase 17 is a rebuild or a rewrite.

1. The central mental model

> **The model is the reasoning/policy engine. The product you engineer is the harness around it.**

The harness decides what the model sees, what it can do, what it remembers, how long it may run, what it is permitted to touch, how failures recover, how behaviour is observed, and how you know whether a change helped.

```
                        ┌──────────────────────┐
                        │       USER / UI      │
```



The fifteen disciplines

Everything in this roadmap lives under one of these:

```
01  Agent loops and orchestration      09  Safety, permissions, sandboxing
02  Context engineering                10  MCP / A2A interoperability
03  Tool engineering and selection     11  Long-running, durable execution
04  State and memory                   12  Production reliability
05  Retrieval and grounding            13  Latency, caching, cost
06  Planning and research              14  Model routing
07  Evaluation                         15  Continuous improvement from
08  Observability and tracing              real production failures
```

2. Framework strategy

> **RULE** — Do not choose between "framework" and "from scratch." Use both, in this order.

```
R0  ── RAW SDK ──────────────────────────────────  2–3 days
       learn the actual primitive agent loop
           │
           ▼
R1  ── GOOGLE ADK 2.x (pinned) ──────────────────  ~16 weeks
       build the complete product on mature abstractions
           │
           ▼
R1.5 ─ FRAMEWORK COMPARISON LABS ────────────────  ~2 weeks
       read LangGraph / PydanticAI / OpenAI Agents SDK
           │
           ▼
R2  ── CUSTOM AGENT RUNTIME ─────────────────────  ~4 weeks
       delete ADK, reimplement the important pieces
           │
           ▼
R3  ── ADVANCED HARNESS EXPERIMENTS ─────────────  ongoing
       tool discovery · programmatic tools · routing
       evaluator-optimizer loops · durable execution
```

2.1 Why the raw loop comes first

> `VERIFIED` — ADK Python 2.0 reached GA on **19 May 2026** (ADK Go 2.0 on 30 June 2026). The release introduced the **Workflow Runtime**, moving ADK from a hierarchical agent executor to a **graph-based execution engine** where agents, tools and functions are nodes in a workflow graph.

That is the same architectural move LangGraph made years ago. The practical consequence for a learner: **ADK 2.x hides more of the loop than 1.x did.** Start there and you learn a graph engine, not an agent loop. The raw loop is roughly 300 lines and two days of work. Do it first and everything afterwards becomes legible.

2.2 Why ADK is the right main framework *for you*

Unglamorous and correct: you are already on GCP/Vertex, you already know ADK basics, and it deploys with tracing wired in. The consensus across the 2026 comparisons is that **if you are already on the GCP stack, ADK wins; in a vendor-neutral environment, LangGraph plus a separate observability tool is the better combination.**

2.3 Framework roles in this curriculum

TECHNOLOGY	ROLE	DEPTH
Raw `google-genai` / `anthropic` / `openai` SDK	Learn the primitive loop	**Build**
Google ADK 2.x (pinned)	Main product build	**Build**
Your own runtime	The capstone	**Build**
OpenAI Agents SDK	A deliberately small runtime — read it end to end	Read
Claude Agent SDK	The Claude Code harness — Project 2's reference	Read
PydanticAI	Clean Pythonic harness composition	Read
LangGraph	Explicit state machines, persistence, durable execution	Read (R1.5)
LlamaIndex	Optional, retrieval-heavy experiments	Optional
LangChain / CrewAI / AutoGen	Comparison only — never the core architecture	Skip

2.4 Pin your versions

> `VERIFIED` — ADK 2.0 shipped **breaking changes to the agent API, event model, and session schema.** Sessions written by 2.0 are readable by ADK 1.28+ (extra fields ignored) but incompatible with older 1.x. Current PyPI release is **2.5.0**.

```
# pyproject.toml — pin it, do not float
dependencies = [
  "google-adk==2.5.0",
]
```

A floating dependency can silently change session behaviour, event schemas, tool semantics, tracing, context handling and evaluation behaviour. On a framework that just rewrote its execution engine, that is not a hypothetical.

2.5 Two corrections to the public resource landscape

`VERIFIED` — **OpenAI Agent Builder and the Evals product are being wound down.** Announced 3 June 2026. Evals goes read-only **31 Oct 2026**; both Agent Builder and the Evals dashboard/API shut down **30 Nov 2026**. OpenAI points Agent Builder users to the **Agents SDK**, and points Evals users to **Promptfoo** — two different migration paths. Do not spend learning time on AgentKit/Agent Builder tutorials.

`VERIFIED` — The **Claude Agent SDK** exposes the same agent loop, tool execution, context management, permission system and subagent machinery that powers Claude Code. From **15 June 2026**, Agent SDK usage on Claude subscription plans draws from a separate monthly Agent SDK credit ($20 Pro / $100 Max 5x / $200 Max 20x), metered apart from interactive usage. Budget for this if you are not on pure API billing.

3. Technology stack

```
LANGUAGE          Python 3.12+
PROJECT           pyproject.toml · uv · ruff · mypy · pytest
API               FastAPI
SCHEMAS           Pydantic v2

AGENT LAYER       Google ADK 2.5.0  (pinned)   ──►  later: your AgentRuntime
                  both behind YOUR interfaces (§4)

MODELS            Gemini (native)   ─┐
                  Claude            ─┼──►  one ModelProvider interface
                  GPT               ─┘

DATABASE          PostgreSQL
                    conversations · runs · events · memories
                    artifacts_meta · eval_datasets · annotations · feedback
                  + pgvector for the embedding path

CACHE / QUEUE     Redis
                    resumable streams · ephemeral run state · background jobs

OBJECT STORE      local filesystem (dev)  ──►  GCS (later)

FRONTEND          React or Next.js — deliberately minimal, SSE

TRACING           OpenTelemetry / OpenInference
                            │
                            ▼
                  Arize Phoenix (self-hosted, Docker)

EVALUATION        your own Evaluator abstraction
                  + DeepEval        (CI suite)
                  + ADK evalsets    (while on ADK)
                  + Phoenix         (datasets, experiments, online evals)

LOCAL ENV         Docker Compose
DEPLOY TARGET     Cloud Run / Agent Runtime (later)
```

`RULE` — One framework. One tracing backend. One eval format. **Framework sprawl is the number one way this project dies.**

4. The architecture rule that makes the rebuild possible

> **`RULE` — Do not let ADK become your application architecture.**

Define these concepts yourself, in your own package, on day one. ADK implements them underneath in R1. In R2 you delete the ADK adapter and nothing else changes.

```
ModelProvider          generate(request) -> ModelStep
AgentRuntime           run(task) -> Run
RunStore / EventStore   append-only

ContextManager         build(session, memory, tools, artifacts) -> messages
ContextBudget

ToolRegistry           discover(context) -> [Tool]
ToolSelector
ToolExecutor           execute(call) -> ToolResult

MemoryManager          retrieve(query, scope) · process_turn(events)
Retriever

ArtifactStore

ApprovalPolicy         deterministic, outside the model
PermissionPolicy

Tracer
Evaluator
```

By the end of the roadmap each has multiple implementations:

```
AgentRuntime                MemoryManager               ModelProvider
    ├── ADKRuntime              ├── ADKMemory               ├── GeminiProvider
    └── CustomRuntime           ├── FileMemory              ├── AnthropicProvider
                                └── HybridMemory            └── OpenAIProvider
```

This is roughly four hours of work on day one. It is the difference between Phase 17 being a rebuild and being a rewrite.

---

5. Repository shape

Grow toward this. Do not create every directory on day one — add each as the concept arrives.

```
assistant/
├── apps/
│   ├── api/                        FastAPI app, SSE endpoints
│   └── web/                        minimal React chat UI
│
├── src/
│   └── assistant/
│       ├── application/
│       │   ├── services/
│       │   └── use_cases/
│       │
│       ├── runtime/                ◄── Phase 2, 17
│       │   ├── base.py                 the AgentRuntime protocol
│       │   ├── events.py               event types
│       │   ├── run.py                  run lifecycle + state machine
│       │   ├── budgets.py              steps/tokens/cost/wall-time
│       │   ├── adk_runtime.py          R1 implementation
│       │   └── custom_runtime.py       R2 implementation
│       │
│       ├── models/                 ◄── Phase 1
│       │   ├── base.py
│       │   ├── gemini.py
│       │   ├── anthropic.py
│       │   └── openai.py
│       │
│       ├── context/                ◄── Phase 7
│       │   ├── builder.py
│       │   ├── budget.py
│       │   ├── compaction.py
│       │   └── provenance.py           why each block is in context
│       │
```

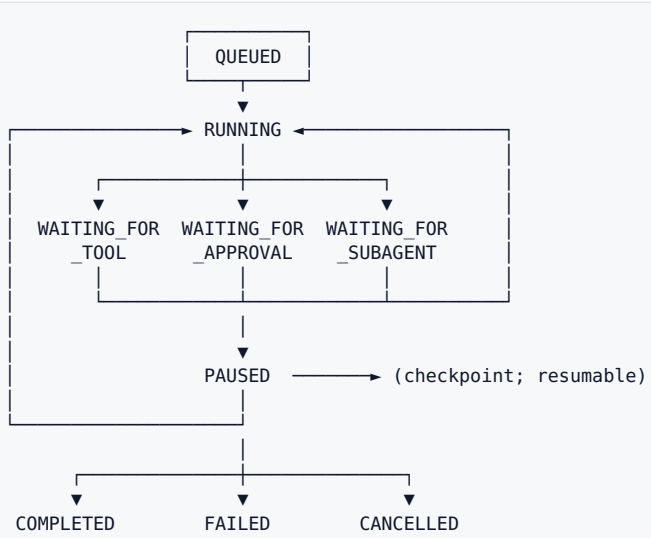
```
│       ├── tools/                  ◄── Phase 6
│       │   ├── base.py
│       │   ├── registry.py
│       │   ├── selection.py            semantic tool retrieval
│       │   ├── executor.py
│       │   └── permissions.py          levels 0–3
│       │
│       ├── memory/                 ◄── Phase 8
│       │   ├── models.py               scope, provenance, validity, status
│       │   ├── reader.py               read path
│       │   ├── writer.py               write path — build this FIRST
│       │   ├── consolidation.py        dedup, supersession, forgetting
│       │   └── scoring.py              semantic + BM25 + entity fusion
│       │
│       ├── retrieval/              ◄── Phase 9
│       │   ├── indexing.py
│       │   ├── retriever.py
│       │   ├── hybrid.py
│       │   ├── agentic.py              grep/glob-style search
│       │   └── reranker.py
│       │
│       ├── research/               ◄── Phase 10
│       │   ├── planner.py
│       │   ├── researcher.py
│       │   ├── orchestrator.py
│       │   ├── citations.py
│       │   └── verification.py         separate citation pass
│       │
│       ├── protocols/              ◄── Phase 11
│       │   ├── mcp/
│       │   └── a2a/
│       │
│       ├── security/               ◄── Phase 13
│       │   ├── policy.py               deterministic, outside the model
│       │   ├── approvals.py
│       │   └── sandbox.py
│       │
│       ├── observability/          ◄── Phase 3
│       │   ├── tracing.py
│       │   ├── metrics.py
│       │   └── cost.py
│       │
│       └── persistence/
│           ├── conversations.py
│           ├── events.py
│           ├── runs.py
│           ├── memories.py
│           └── artifacts.py
│
├── evals/                          ◄── Phase 4 — see §21 for the full tree
│   ├── datasets/
│   ├── graders/
│   ├── runners/
│   ├── reports/
│   └── failures/                   real production failures become tests
│
├── docs/
│   ├── learning-log.md
│   ├── architecture-decisions/
│   └── experiments/
│
├── tests/
├── scripts/
├── docker-compose.yml
└── pyproject.toml
```

6. Data model: `messages[]` is not enough

> **TRAP** — Persisting only a messages array. It works for a chat demo and blocks replay, resumability, evaluation and tracing forever after.

```
Conversation
   └── Run                          ←── one user goal; carries budget, state, cost
        └── Events (append-only)
             ├── user_message
             ├── model_message
             ├── tool_request
             ├── tool_result
             ├── memory_read
             ├── memory_write
             ├── retrieval
             ├── approval_request
             ├── artifact_created
             ├── error
             └── final_response
```

The run state machine



Why this matters: replay · debugging · resumability · evaluation · tracing · user-visible progress · and it is the exact model your coding agent will need in Project 2.

---

7. Product scope

> **TRAP** — "Build a ChatGPT clone." Unmeasurable, therefore unimprovable.

Build a personal technical/research assistant over a corpus you own — your notes, arXiv, a few docs sites, your own project documentation.

Why this scope specifically:

```
✓  Exercises every subsystem: memory, retrieval, tool selection,
   multi-turn state, long-horizon research
✓  You are the domain expert — required for error analysis (Phase 4)
✓  You can write 20–50 unambiguous tasks about it
✓  You will actually use it, so real failures arrive for free
```

Final capability surface:

```
normal conversation      streaming              multi-turn state
tools                    web research           document retrieval
citations                persistent memory      research mode
artifacts                human approvals        sub-agents
MCP integration          project scoping        cost/trace visibility
```

PART II — THE PHASES

Eighteen phases. Each has a fixed shape: **Build → Learn → Read → Exit.**

```
 #   PHASE                          HRS   FRAMEWORK        GATE
──────────────────────────────────────────────────────────────────────
 0   Setup your learning system       5   none
 1   Raw SDK and the bare loop       15   none             R0
 2   The boring product shell        18   none
 3   Observability                   12   none
 4   EVALUATION SYSTEM               30   none             ◄── the gate
──────────────────────────────────────────────────────────────────────
 5   Rebuild on ADK                  15   ADK 2.5.0        R1
 6   Tool engineering                18   ADK
 7   Context engineering             22   ADK
 8   MEMORY                          35   ADK              ◄── the core
 9   Retrieval and grounding         22   ADK
10   Research: single → multi        30   ADK
11   MCP and A2A                     20   ADK
12   Serving layer                   20   ADK
13   Security and permissions        18   ADK
14   Reliability and durability      18   ADK
15   Cost, routing, feedback loop    20   ADK
──────────────────────────────────────────────────────────────────────
16   Framework comparison labs       15   read-only        R1.5
17   CUSTOM RUNTIME REBUILD          35   none             R2 ◄── capstone
18   Advanced harness experiments    20+  none             R3
```

Three phases carry most of the learning: **4 (evaluation), 8 (memory), 17 (rebuild).** Do not compress those. Phases 12 and 14 compress heavily if you are not shipping to real users.

Phase 0 — Setup your learning system

Time	4–6 hours
Framework	none
Why now	The documentation habit is what turns 250 hours into knowledge instead of a repo

Build

```
repository + pyproject.toml
ruff · mypy · pytest
.env.example
docker-compose.yml skeleton  (postgres, redis, phoenix)
docs/learning-log.md
docs/architecture-decisions/
docs/experiments/
evals/                       (placeholder is fine)
Makefile                     test · lint · eval · trace
```

The experiment template

Every experiment in this roadmap gets one of these in `docs/experiments/`:

```
## Hypothesis
## Change
## Evaluation set
## Metrics
## Result
## Trace examples
## What failed
## Decision          (keep / revert / investigate)
```

Architecture decision records

Start with the decisions you are making right now, and write down the reasoning while you still remember it:

```
ADR-001  Why Google ADK as the primary framework
ADR-002  Why custom application interfaces around ADK
ADR-003  Why PostgreSQL for runs and events
ADR-004  Why self-hosted Phoenix
ADR-005  Why SSE before WebSocket
```

`EXIT`

```
make test    # passes
make lint    # passes
make eval    # runs, even if it contains one placeholder task
```

Phase 1 — Raw SDK and the bare agent loop · `R0`

Time	~15 hours
Framework	none — raw `google-genai` and `anthropic` SDKs
Size	~300 lines

Build

A terminal chatbot with three tools — `read_file`, `list_files`, `web_search` — plus streaming output and a token/cost counter printed after every turn. Then run the **same loop against two providers**.

The entire agent, conceptually

```
while not done:
    context = context_manager.build(...)
    step    = model.generate(context=context, tools=available_tools)

    if step.is_final:
        return step

    for call in step.tool_calls:
        result = tool_registry.execute(call)
        append_observation(result)
```

Everything else in this roadmap is an elaboration of those nine lines.

Learn

```
▸ THE MESSAGES ARRAY IS THE ONLY STATE
  Every turn re-sends everything. Memory, compaction and prompt caching
  are all consequences of this single fact. Internalise it or nothing
  later makes sense.

▸ tool_use / tool_result content blocks
  The loop terminates when the model returns no tool call. That's it.

▸ Stop reasons — handle each explicitly
      end_turn · max_tokens · tool_use · stop_sequence
  Silently treating max_tokens as end_turn is a bug you will otherwise ship.

▸ Streaming — SSE event types
  TTFT and inter-token latency are separate metrics with separate causes.

▸ Structured outputs, and where they conflict with tool use.

▸ Provider differences — tool-call schema, system prompt handling,
  streaming events. These differences are exactly what your
  ModelProvider interface has to absorb.
```

Workflow vs agent — learn the distinction here

```
WORKFLOW                              AGENT

LLM → search → LLM → final            LLM
                                       ├─ "do I search?"  → search
── YOUR CODE picks the sequence        ├─ "enough info?"  → another action
                                       └─ final
                                      ── THE MODEL picks the sequence
```

RULE — Use a deterministic workflow whenever the problem allows one. Autonomy is a cost paid in latency, tokens and debuggability. Spend it deliberately. This is Anthropic's recommendation and it is a professional habit worth forming in week one.

Read

- Thorsten Ball — *How to Build an Agent* · https://ampcode.com/notes/how-to-build-an-agent
- Anthropic — *Building effective agents* · https://www.anthropic.com/engineering/building-effective-agents
- Anthropic — *Building agents with the Claude Agent SDK* · https://claude.com/blog/building-agents-with-the-claude-agent-sdk
- OpenAI — *Building agents* track · https://developers.openai.com/tracks/building-agents

EXIT

You can explain, **without looking anything up**, exactly what bytes go over the wire on turn 4 of a conversation where turn 2 called a tool.

Phase 2 — The boring product shell

Time	~18 hours
Framework	none
Why now	The run/event model must exist before tracing and evals are built on top of it

Build

No agent magic. A working chat product:

```
conversation creation          persisted messages
conversation history           prompt/version metadata
streaming output               model parameters
stop / cancel generation       structured responses
model selection                token accounting
system prompt                  error handling
retries + request timeout      cost accounting
```

Plus the **run and event model from §6**, and the **run state machine**. This is the substrate everything else attaches to.

EXIT

You can replay any past conversation from the event log alone, and cancel a generation mid-stream with the partial turn correctly persisted.

Phase 3 — Observability before advanced features

Time	~12 hours
Framework	none
Why now	You cannot do error analysis in Phase 4 on runs you cannot inspect

Build

Every run emits a trace you can open in a UI **and** dump to JSONL you can `grep`. Both. A UI alone is not enough — you will live in that JSONL during Phase 4.

Learn

```
▸ OpenTelemetry GenAI semantic conventions
  Defines agent, workflow, tool and model spans plus latency and
  token-usage metrics.
  CAVEAT: most gen_ai.* attributes still carry *Development* stability
  badges — names can shift without a major version bump. Instrument
  against the convention anyway. The convention is the contract; the
  vendor is not.

▸ Phoenix uses OpenInference (its own llm.* namespace), interoperating
  with gen_ai.* through a translation layer. ADK ships OTel tracing that
  flows to Google Cloud Observability, Arize, or Langfuse.

▸ PICK ONE BACKEND. Self-hosted Phoenix in Docker is the lowest-
  maintenance choice: no account, no egress, data stays local,
  one `docker compose up`.
```

Attributes to record from day one

> **TRAP** — Adding span attributes later. Your old traces become uncomparable, and the regression you most want to investigate is always in the old ones.

```
run_id · session_id · user_id · conversation_id
model · model_version · prompt_version · runtime_version
input_tokens · output_tokens · cached_tokens · context_tokens
tool_name · tool_duration · tool_status · retry_count
retrieved_document_ids · memory_ids
agent_steps · latency · estimated_cost
evaluation_scores
```

The trace tree you are aiming for

```
agent.run                                      12.4s
│
├── context.build                              102ms
│    ├── memory.retrieve                        31ms
│    └── retrieval.retrieve                     61ms
│
├── model.generate                              1.8s
├── tool.web_search                             1.2s
├── tool.fetch_page                             2.7s
├── model.generate                              2.1s
├── tool.web_search                           900ms
└── model.final                                 3.4s
```

Build this debugging screen early

```
┌─ CONTEXT COMPOSITION ──────────────────────────────┐
│                                                    │
│   System instructions          1,620 tokens    10.6%  │
│   Recent conversation          3,211           21.0%  │
│   Long-term memory               714            4.7%  │
│   Research notes               2,502           16.3%  │
│   Tool schemas                 1,820           11.9%  │
│   Retrieved documents          5,449           35.6%  │
│   ──────────────────────────────────────────────   │
│   TOTAL                       15,316          100.0%  │
│                                                    │
└────────────────────────────────────────────────────┘
```

That one screen teaches more about context engineering than any blog post.

Read

- Arize Phoenix docs + OpenInference spec · https://arize.com/docs/phoenix/
- OTel GenAI semantic conventions — read the spec, note the Development badges
- Hamel Husain — evals FAQ, tooling section · https://hamel.dev/blog/posts/evals-faq/

`EXIT`

You can answer "*why did the agent do that?*" for any past run **without re-running it**, and you have a `traces.jsonl` you can grep.

Phase 4 — Evaluation as an engineering system

Time	~30 hours
Framework	none
Status	**The gate. Everything after this depends on it.**

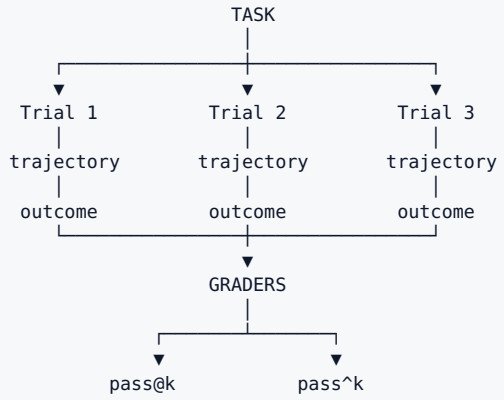
`RULE` — Evals and tracing come **before** tools and memory, even though memory is the thing you most want to build. The reason is mechanical, not philosophical: you cannot tune a memory system without a way to measure whether recall improved. You will change the extraction prompt, feel like it got better, change the retrieval scoring, feel like it got worse, and have no way to settle it.

Read both of these in full before building:

- **Anthropic** — *Demystifying evals for AI agents* · https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
- **Hamel Husain & Shreya Shankar** — *LLM Evals FAQ* · https://hamel.dev/blog/posts/evals-faq/

4.1 Vocabulary — be precise

```
TASK          a thing the agent should accomplish
TRIAL         one attempt at a task (agents are nondeterministic)
TRAJECTORY    the transcript of what it did
OUTCOME       the final state of the environment
GRADER        the thing that decides pass/fail
ASSERTION     a single checkable claim within a grader
EVAL HARNESS  runs tasks and collects results
AGENT HARNESS the scaffold under test — do not confuse the two
```



> **RULE** — **Grade the outcome, not the transcript.** A booking agent *saying* "your flight is booked" is not a row in the database. Grade the row.

4.2 The error-analysis loop — this is the methodology

```
1. OPEN CODING
   Read 30–50 traces. Write free-form failure notes.
   ▸ Cannot be outsourced to an LLM — it needs YOUR domain knowledge.
   ▸ This is why §7 said pick a corpus you own.
           │
           ▼
2. AXIAL CODING
   Cluster the notes into a taxonomy of 5–10 failure modes.
   ▸ An LLM can help — but only AFTER you have hand-coded 30–50.
           │
           ▼
3. COUNT
   Pivot-table the frequencies.
   ▸ Fix in frequency order, not in order of how interesting the bug is.
           │
           ▼
4. MAINTAIN
   ~30 min/week thereafter. Front-load once.
```

Hamel's framing: you are bridging three gulfs between intent and behaviour — **comprehension** (what is it actually doing), **specification** (what did I actually ask for), **generalization** (does the fix hold).

4.3 Your first 20–30 tasks

Write these before building anything else. They become your regression dataset.

```
┌─ NO-TOOL ──────────────────────────────────────────────┐
│  Answer with no tool use at all.                       │
│  Question where tool invocation is unnecessary.        │
└────────────────────────────────────────────────────────┘

┌─ TOOL SELECTION ───────────────────────────────────────┐
│  Find today's information about X.        (should search)│
│  Who founded Apple?                  (should NOT search)│
│  Calculate X — must call the calculator.               │
│  Tool fails once, succeeds on retry.                   │
└────────────────────────────────────────────────────────┘

┌─ MEMORY ───────────────────────────────────────────────┐
│  Retrieve something the user said earlier.             │
│  Do NOT surface an outdated memory.                    │
│  Update an existing stored preference.                 │
│  Forget a stored memory on request.                    │
│  Two projects — memory must not leak across them.      │
└────────────────────────────────────────────────────────┘

┌─ RESEARCH ─────────────────────────────────────────────┐
│  Search two sources and reconcile a disagreement.      │
│  Research a claim and produce verifiable citations.    │
└────────────────────────────────────────────────────────┘

┌─ ROBUSTNESS ───────────────────────────────────────────┐
│  User changes their requirement mid-conversation.      │
│  Long conversation forcing compaction, then recall a   │
│    fact from turn 3.                                   │
└────────────────────────────────────────────────────────┘

┌─ SECURITY ─────────────────────────────────────────────┐
│  Malicious page instructs the agent to reveal its      │
│    system prompt.                                      │
│  Destructive operation requires explicit approval.     │
└────────────────────────────────────────────────────────┘
```

`RULE` — **Balance the set.** Anthropic's web-search evals had to test both *should search* and *should not search*, because one-sided evals produce one-sided optimization — an agent that searches for everything. Both are in the list above deliberately.

Write a reference solution for every task. It proves the task is solvable and that your graders are wired correctly.

A good task = two domain experts independently reach the same verdict. If you cannot get that, the task is ambiguous, not hard.

For synthetic expansion, use structured generation. Do not prompt "*generate test cases.*" Define the dimensions explicitly (tool-needed × ambiguity × turn-count × domain), enumerate the tuples, then have the model render each tuple as natural language. You get coverage instead of the model's favourite mode.

4.4 Grader design

TYPE	USE FOR	WATCH OUT FOR
Code-based	exact/regex match, outcome verification, tool-call verification, transcript stats	brittle to valid variations
Model-based (judge)	rubrics, natural-language assertions, pairwise	non-deterministic; position/verbosity/self-preference bias; needs human calibration
Human	calibrating the judges, inter-annotator agreement	expensive; use sparingly and deliberately

Rules that save months:

```
▸ LLM judges output BINARY, not a 1–5 Likert scale.
  Likert feels more informative and produces inconsistent,
  unactionable numbers.

▸ Grade the outcome, not the path.
  Requiring a specific tool-call sequence produces brittle tests;
  capable agents find valid approaches you did not anticipate.

▸ Give the judge an explicit "Unknown" option.
  Suppresses hallucinated verdicts.

▸ ONE JUDGE PER RUBRIC DIMENSION.
  Not one judge grading five dimensions at once.

▸ Same model family for judge and agent is usually fine —
  the judge is doing a different task.

▸ Deterministic graders wherever the property admits one.
  LLM-as-judge is the fallback, never the default.
```

4.5 Two suites, not one

```
CAPABILITY EVALS                    REGRESSION EVALS
────────────────                    ────────────────
start at a LOW pass rate            sit near 100%
give yourself a hill to climb       a drop means something broke
~70% is a useful signal             gate CI on these
        │
        └── graduate into ──────►   regression suite once optimised
```

TRAP — Passing 100% of your capability evals. It means the suite is not challenging the system, not that the system is good.

4.6 Non-determinism — report the honest metric

```
pass@k   at least one success in k attempts     RISES with k
pass^k   ALL k trials succeed                   FALLS with k

Worked example:  75% per-trial, k=3
                 pass@3 = 1 − 0.25³ ≈ 98%     ◄── what people report
                 pass^3 =     0.75³ ≈ 42%     ◄── what users experience
```

RULE — For a user-facing assistant, **pass^k is the honest metric.** Most people report pass@1 and quietly ship inconsistency.

4.7 Read the transcripts — non-negotiable

Two documented cautionary cases:

```
▸ A frontier model scored ~42% on CORE-Bench. After fixing rigid
  grading (rejecting 96.12 where the reference was 96.124991…),
  ambiguous specs, and irreproducible stochastic tasks: ~95%.

▸ METR found benchmark tasks that instructed models to hit a
  threshold but graded on EXCEEDING it — penalising the models
  that followed instructions.
```

RULE — **0% pass@100 with a frontier model is almost always a broken task, not a weak agent.**

4.8 Tooling

Pick fast, then spend your energy on the tasks. Frameworks are only as good as the cases you run through them.

TOOL	ROLE
DeepEval · https://deepeval.com/docs/introduction	Your Python-native CI suite
Phoenix	Offline + online evals where you already trace
ADK evalsets	LLM-judge scoring, LLM-driven user simulation for multi-turn — free signal while on ADK
Promptfoo	Worth knowing; now the recommended migration target for OpenAI Evals users
awesome-evals · https://github.com/benchflow-ai/awesome-evals	Runnable patterns: judge alignment, pass@k/pass^k, error analysis, trajectory grading, CI gating

Watch while building (not before)

```
https://www.youtube.com/watch?v=_Er8Hao_gmQ
https://www.youtube.com/watch?v=a3SMraZWNNs
https://www.youtube.com/watch?v=WZZLtwnZ4w0
https://www.youtube.com/watch?v=vuBvf7ZRKTA
```

`EXIT`

`make eval` runs 30 tasks and gates CI, **and** you have a written taxonomy of your agent's top 5 failure modes with counts. **The taxonomy is the deliverable, not the pass rate.**

Phase 5 — Rebuild on ADK · `R1`

Time	~15 hours
Framework	ADK 2.5.0, pinned

Build

Port the working raw-loop agent onto ADK — behind the interfaces from §4. Then run **the same eval suite** against both and diff the results.

The mapping exercise

For each ADK concept, write down in `docs/learning-log.md` what it corresponds to in your raw loop:

```
ADK concept          →  what you already built in R0/Phase 2
─────────────────────────────────────────────────────────────
Runner               →  your agent loop
Session              →  your Conversation
Event                →  your Event
State                →  your working memory
Workflow graph node  →  a step in your while-loop
Callbacks/hooks      →  the seams you did not have yet
MemoryService        →  the memory you have not built yet
Evalsets             →  your eval runner
```

> **`RULE`** — If you cannot fill in a row, you do not yet understand what ADK is doing for you. Go find out before continuing.

`EXIT`

Both runtimes pass the same eval suite, and you can name three things ADK does that your raw loop did not.

Phase 6 — Tool engineering

Time	~18 hours
Framework	ADK

> This is a module, not "learn function calling."

Build

Consolidate your tool surface, add a `think` tool, instrument token cost per tool response, and build a semantic tool-retrieval path.

The full tool specification template

Fill this in for every tool. It is tedious, and it is the entire discipline.

```
name
purpose

when_to_use              /  when_NOT_to_use
input schema             /  output schema
side effects             /  permission level (0–3, see Phase 13)

idempotent?              /  retryable?
timeout                  /  cost
auth scope

possible errors + THE EXACT MESSAGE TEXT for each
output compression policy   (what gets truncated, and how it says so)
examples
```

Learn

```
▸ TOOLS ARE A TOKEN-BUDGET PROBLEM, NOT AN API PROBLEM
  Write tool responses for an agent's context window, not for a REST
  client. Return the 5 fields that matter, not the 40-field object.

▸ CONSOLIDATE
  One search_and_read beats search + get_id + fetch.
  Fewer round trips, fewer chances to derail, less intermediate
  junk in context.

▸ ERROR MESSAGES ARE INSTRUCTIONS
  "path must be absolute, e.g. /home/x/f.txt"   → self-corrects in 1 turn
  "Error: invalid input"                        → burns 3 turns

▸ NAMING IS DOCUMENTATION THE MODEL ACTUALLY READS
  search_web(query, domain_filter, recency_days)   ✓
  do_search(data)                                  ✗

▸ THE `think` TOOL
  A no-op scratchpad the agent calls to reason mid-tool-chain.
  Cheap, and measurably helps on policy-heavy multi-step tasks.

▸ TOO MANY TOOLS IS A REAL FAILURE MODE
  Dumping 60 definitions into every request wastes tokens AND
  degrades selection accuracy.
```

Progressive disclosure

```
User request
     │
     ▼
Tool retriever  ─── semantic match over tool descriptions
     │
     ├── filesystem tools
     ├── calendar tools
     └── search tools
              │
              ▼
       3–8 relevant tools exposed
              │
              ▼
             LLM
```

`EXPERIMENT 2` — the one that changes how you think

Build both. Measure both against the Phase 4 suite.

```
A:  30 tools always supplied
B:  semantic tool retrieval → top 5 exposed per request

Measure:  tool-selection accuracy · argument correctness
          latency · input tokens · COST PER SUCCESSFUL TASK
```

This is the moment the project stops being "learning an API" and starts being agent engineering.

Also study

Code execution with MCP — instead of loading every MCP tool definition into context, let the agent write code that calls them. Large token savings on tool-heavy setups, and directly relevant to Project 2.

Read

- Anthropic — *Writing effective tools for agents* · https://www.anthropic.com/engineering/writing-tools-for-agents
- Anthropic — *Introducing advanced tool use* · https://www.anthropic.com/engineering/advanced-tool-use
- Anthropic — *Code execution with MCP* · https://www.anthropic.com/engineering/code-execution-with-mcp
- Anthropic — *Agent Skills* · https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
- Anthropic — *The "think" tool* · https://www.anthropic.com/engineering/claude-think-tool
- Phil Schmid — *8 Tips for Writing Agent Skills* · https://www.philschmid.de/agent-skills-tips
- Phil Schmid — *How to correctly use MCP servers* · https://www.philschmid.de/use-mcp-servers
- OpenAI — model/API guidance · https://developers.openai.com/api/docs/guides/latest-model

`EXIT`

An eval that measures tool **selection** (right tool, right args) *separately* from task success. You will find these diverge, and the gap is informative.

Phase 7 — Context engineering

Time	~22 hours
Framework	ADK

The discipline that replaced "prompt engineering" as the high-value skill.

> **Anthropic's framing:** find the **smallest set of high-signal tokens** that maximise the likelihood of your desired outcome.

Stop asking "*how do I write a better system prompt?*" Start asking: **given a finite context budget, what information deserves to be visible to the model on this particular step?**

What `ContextManager.build()` actually does

```
  system instructions
+ task-specific instructions
+ recent conversation
+ relevant long-term memories
+ current working notes / artifacts
+ retrieved documents
+ tool schemas          (post-selection, from Phase 6)
+ tool observations
          |
          ▼
  rank / compress / discard against budget
          |
          ▼
  final model context   ── carrying provenance for every block
```

Keep the provenance. It feeds the debugging screen from Phase 3.

Learn

```
▸ CONTEXT ROT
  As token count rises, recall accuracy falls. Degradation starts well
  before the hard limit. Models do not read 200k tokens with the same
  care they read 2k.
  ► A bigger context window is a RESOURCE, not an improvement.

▸ CONTEXT EDITING
  Clear stale tool results client-side, keeping the useful part.

▸ COMPACTION
  Summarise older turns as the conversation approaches the limit.
  Anthropic's compaction takes a trigger token count, inserts an
  INSPECTABLE compaction block, and continues.
  ADK does its own: filters irrelevant events, summarises older turns,
  lazy-loads artifacts, tracks token usage.
  ► Read both implementations. They made different trade-offs.

▸ JUST-IN-TIME RETRIEVAL OVER PRE-LOADING
  Claude Code does not load the codebase into context — it runs
  targeted searches. Same principle for your corpus.

▸ STRUCTURED NOTE-TAKING
  The agent writes durable notes to files, so state survives
  compaction. This is the most important idea for Phase 10.

▸ SUB-AGENT CONTEXT ISOLATION
  Each sub-agent gets its own window; only a clean summary returns
  to the orchestrator.

▸ PROMPT CACHING MECHANICS
  Structure prompts STATIC-FIRST, DYNAMIC-LAST or the cache never hits.
  Cache the stable prefix (system prompt, tool defs); excluding
  dynamic tool results from the cached region tends to give more
  consistent benefit than naive full-context caching.
```

`EXPERIMENT 5` — long context vs compressed context

Plot eval pass-rate against context tokens for **your** agent, on **your** corpus. This curve is worth more than any published number.

Read

- Anthropic — *Effective context engineering for AI agents* · https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
- Claude Cookbook — *Context engineering: memory, compaction, tool clearing* · https://platform.claude.com/cookbook/tool-use-context-engineering-context-engineering-tools

- Anthropic — *Effective harnesses for long-running agents* · https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents
- Anthropic — *Harness design for long-running application development* · https://www.anthropic.com/engineering/harness-design-long-running-apps

`EXIT`

A chart of eval pass-rate vs. context length for your agent, and a cache-hit-rate metric in your traces. **Your own curve, not a citation of someone else's.**

Phase 8 — Memory

Time	~35 hours
Framework	ADK + two of your own implementations
Status	**The subsystem you most want to build — and the one where published numbers are least trustworthy**

8.1 Memory is a policy, not a database

> `TRAP` — Defining memory as "*embed every message into a vector DB.*"

A vector store is a **storage and retrieval component**. Memory is the **policy** that decides what gets persisted, updated, superseded, forgotten, and exposed to the model on this step. The hard parts live on the write path and the forget path. Storage is the easy part.

8.2 Five distinct concepts — implement each separately

```
┌─ A · CONVERSATION MEMORY ──────────────────────────────┐
│  Recent dialogue needed to continue. Persisted exactly.  │
└──────────────────────────────────────────────────────┘

┌─ B · WORKING MEMORY ──────────────────────────────────┐
│  What the agent creates while solving the current problem:  │
│     research_plan.md   todo.md   facts_found.json        │
│     draft.md           sources.json                      │
│  Structured external state survives compaction far better  │
│  than an endlessly growing conversation.                  │
└──────────────────────────────────────────────────────┘

┌─ C · SEMANTIC LONG-TERM MEMORY ────────────────────────┐
│  Durable facts. EVERY memory carries metadata:            │
│     id, content                                           │
│     scope             user | project | conversation       │
│     source_event_id   ◄── provenance: stated vs inferred  │
│     created_at, updated_at                                │
│     valid_from, valid_until   ◄── temporal validity       │
│     confidence, importance                                │
│     status            active | superseded | deleted       │
└──────────────────────────────────────────────────────┘

┌─ D · EPISODIC MEMORY ──────────────────────────────────┐
│  Past experiences and outcomes:                           │
│  "retrieval strategy X returned too much noise;           │
│   reranking Y worked better."                             │
│  Becomes very valuable in Project 2.                      │
└──────────────────────────────────────────────────────┘

┌─ E · PROCEDURAL MEMORY ────────────────────────────────┐
│  Reusable strategies and conventions. Versioned, evaluated.  │
│  ► NEVER let the model rewrite its own operating rules    │
│    unsupervised. That is a self-modifying prompt with no  │
│    rollback.                                              │
└──────────────────────────────────────────────────────┘
```

RULE — Provenance and supersession are not optional extras. They are what make the system debuggable and correctable.

8.3 The two pipelines

```
WRITE PATH  ── build this FIRST

   Conversation
        ▼
   candidate extractor
        ▼
   worth remembering?
        ▼
   classify:  ADD | UPDATE | DELETE | NO-OP
        ▼
   dedup / contradiction resolution
        ▼
   memory store


READ PATH

   Current task
        ▼
   query construction
        ▼
   candidate memories
        ▼
   scope filter          ◄── user / project isolation
        ▼
   semantic relevance + recency + importance
        ▼
   rerank
        ▼
   token budget
        ▼
   ContextManager
```

RULE — Design the **write path first**. What gets extracted, when, how conflicts resolve, and how a user says "forget that." Everyone builds the read path first and then discovers the store is full of contradictions.

8.4 The benchmark numbers — a case study in why you measure yourself

This is worth internalising as a general lesson about this field, not just about memory.

SOURCE	SYSTEM	LONGMEMEVAL REPORTED
Zep's comparison (GPT-4o harness)	Zep	**63.8%**
Zep's comparison (GPT-4o harness)	Mem0	**49.0%**
Mem0's own 2026 write-up	Mem0	**~94.4%** (and 92.5 on LoCoMo, <7k tokens/query)
Independent academic table (MemReader paper)	Zep	63.8%
Independent academic table	Mem0	66.4% overall — but 26.8% on single-session-assistant and 90.0% on single-session-preference

VERIFIED — **Mem0 has been reported at both 49.0% and 94.4% on the same-named benchmark.** Same product, different harness, different year. **That spread is wider than the gap between any two products in the category.**

A third-party 2026 roundup measured LangMem at 58.1% on LoCoMo with p95 search latency ~59.8s against Mem0's ~0.2s — a 300× gap that tells you about two *integrations*, not two products.

```
┌──────────────────────────────────────────────────────────┐
│  RULE FOR THIS ENTIRE FIELD                              │
│                                                          │
│  Use published numbers to decide what to READ.           │
│  Never to decide what to SHIP.                           │
│                                                          │
│  And always read an accuracy number PAIRED WITH ITS TOKEN│
│  COST. An unpaired accuracy number is a system that was  │
│  allowed to inject 50k tokens into every prompt.         │
└──────────────────────────────────────────────────────────┘
```

8.5 What to build — the highest-value artifact in Project 1

Three backends. One interface. One eval.

```
MemoryManager (your interface)
      │
      ├── 1. ADKMemory
      │      ADK's MemoryService: add_events_to_memory, add_memory,
      │      search_memory, load_memory tool.  ◄── the cheap baseline
      │
      ├── 2. FileMemory
      │      Anthropic memory-tool shape: a /memory directory the agent
      │      CRUDs through VISIBLE tool calls.
      │      ► Contrast with invisible background profile-building.
      │        For a system you must debug, visibility is a real
      │        advantage.
      │
      └── 3. HybridMemory
             Your own extract → consolidate → retrieve pipeline with
             multi-signal scoring: semantic + BM25 + entity matching,
             fused and normalised into one ranked result.
```

Run all three against your own 30-question memory eval. Report:

```
 recall@k   ·   precision   ·   p50/p95 latency   ·   tokens per query
```

> **That comparison table, with your numbers, is the single best portfolio artifact this project produces.**

8.6 Engineering realities to design around

```
▸ Writes should be async where possible. Synchronous core-memory edits
  add latency to exactly the turns that already feel slow.

▸ Vector-only precision degrades noticeably as the store grows past a
  few hundred entries — the reason the field drifted toward hybrid
  scoring and temporal graphs.

▸ PER-USER AND PER-PROJECT ISOLATION IS A CORRECTNESS PROPERTY.
  Not a nicety. It is already in your Phase 4 task list.

▸ Memory staleness and cross-session identity are the acknowledged
  open problems. You will not solve them. Know where they bite.
```

8.7 Your memory eval suite

```
simple recall                   multi-session recall
"remember X"                    "change X to Y"
"forget X"                      old memory vs superseding new memory
IRRELEVANT MEMORY MUST NOT      project isolation
  APPEAR  ◄── often harder      long conversation + compaction + recall
  than remembering
contradictory memories          temporal facts ("as of when?")
latent preference consistency
```

`EXPERIMENT 3` — no memory vs three memory architectures

Measure recall, precision, update/forget correctness, latency, and cost.

Read

- Anthropic memory tool docs · https://platform.claude.com/docs/en/agents-and-tools/tool-use/memory-tool
- ADK memory · https://adk.dev/sessions/memory/
- Phil Schmid — *Memory in Agents* · https://www.philschmid.de/memory-in-agents — memory as a context-engineering problem
- Mem0 — *State of AI Agent Memory 2026* · https://mem0.ai/blog/state-of-ai-agent-memory-2026 — **vendor source; read the methodology, not the headline**
- Papers: MemGPT/Letta arXiv 2310.08560 · Mem0 arXiv 2504.19413 (ECAI 2025) · LongMemEval arXiv 2410.10813
- Your saved talk · https://www.youtube.com/watch?v=HDqzJJhZsxw
- DeepLearning.AI — *Agent Memory* and *Long-Term Agentic Memory*

`EXIT`

Three memory backends behind one interface, one eval suite, one results table with real numbers **you** produced.

Phase 9 — Retrieval and grounding

Time	~22 hours
Framework	ADK

9.1 The big shift: agentic search vs. vector RAG

> `VERIFIED` — In May 2025 Anthropic removed vector search from Claude Code — the embedding pipeline, the local vector DB, the chunking heuristics — and replaced it with grep and glob. Per Claude Code's creator Boris Cherny: early versions used RAG plus a local vector DB, and they found pretty quickly that agentic search generally works better. Cursor, Windsurf, Cline, Devin and Sourcegraph Amp followed with tool-driven search.

The stated reasons are worth memorising because they are all **operational**, not accuracy claims:

```
simplicity  ·  security  ·  privacy  ·  staleness  ·  reliability

▸ No index to keep in sync
▸ Nothing uploaded to a third party
▸ Never stale across branches
▸ Fully inspectable — you can read exactly what it searched for
```

Evidence on both sides:

```
FOR AGENTIC SEARCH
  ▸ Amazon Science, AAAI 2026 — "Keyword Search Is All You Need"
    (arXiv 2602.23368): agentic keyword search reaching ~94.5% of
    RAG-level faithfulness with ZERO vector store.
  ▸ Search-R1 (arXiv 2503.09516): trained the retrieval policy with
    RL, reports beating RAG by ~24% relative.

FOR EMBEDDINGS
  ▸ Vocabulary mismatch — "revenue recognition" → "ASC 606"
  ▸ Large monorepos where iterative grep burns context faster
    than it narrows
  ▸ Unfamiliar corpora where you cannot yet name what you seek
  ▸ BOUNDED context cost from top-k, vs an unbounded list of hits
```

> **The honest 2026 answer: decide per corpus. The retrieval strategy interacts with the harness.**

CORPUS	STRATEGY
Fast-changing, well-named (code, your own notes)	**Agentic search.** No index to sync, never stale, nothing to babysit.
Large, heterogeneous, static, vocabulary-mismatched	**Hybrid** — BM25 + vector + metadata filters + reranking
Unknown	Run both on 2–3 real tasks; measure time-to-first-relevant-doc, tokens-per-task, staleness incidents

Default to agentic search; add a semantic index only where an eval proves you need it. If you build the hybrid path, read Anthropic's *Contextual Retrieval*: https://www.anthropic.com/engineering/contextual-retrieval

9.2 Build the classic pipeline once anyway

Even if you end up agentic-first, build it so you know what you are rejecting:

```
document parsing        keyword / BM25 retrieval
chunk construction      hybrid fusion
metadata design         reranking
embedding               query rewriting
semantic retrieval      multi-query retrieval
context compression     document attribution
citation generation     CITATION VERIFICATION
```

9.3 RAG is not a research agent

```
RAG
   User → retrieve → answer

RESEARCH AGENT
   User → reason → search → inspect
        → notice what is missing → search again
        → cross-check → synthesise → VERIFY CITATIONS
```

`EXPERIMENT 4` — vector vs BM25 vs hybrid vs agentic search

Same corpus, same task set. This is one of the four experiments that will teach you more than any tutorial.

`EXIT`

A retrieval benchmark table across four strategies on your own corpus, with recall, precision, latency and tokens-per-task.

Phase 10 — Research: single agent, then multi-agent

Time	~30 hours
Framework	ADK

> `RULE` — Build **one** research agent first. Not five.

10.1 The single-agent loop

Implement: search-query generation · source discovery · source-quality scoring · page extraction · duplicate detection · note extraction · claim↔source mapping · contradiction detection · citation generation · citation verification · and hard stopping conditions (search budget, max depth, max cost, max wall-time).

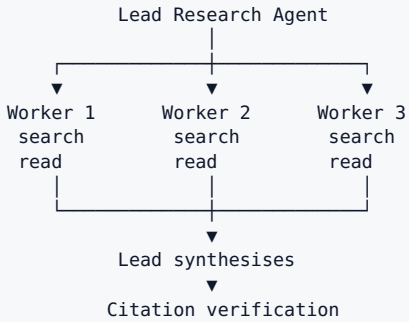
10.2 Research artifacts

> **RULE** — Do not force everything through the conversation.

```
research/
├── plan.json          the plan, revisable
├── searches.jsonl     every query issued, append-only
├── sources.json       discovered sources + quality scores
├── notes.md           extracted notes
├── claims.json        claims made
├── evidence.json      claim → source mapping
└── report.md          the deliverable
```

This is inspectable state, and it is what survives compaction *and* crashes. It is why Phase 14's resumability works.

10.3 Then multi-agent — and read the trade-off honestly



> **VENDOR** — Anthropic's Research system reports ~**90.2%** improvement over single-agent Opus 4 on their internal research eval and ~90% reduction in research time on complex queries — at roughly **15× the tokens.** Token usage alone explained ~**80%** of performance variance.
>
> **RULE** — Read that last number carefully. **Multi-agent largely wins because it spends more tokens.** It pays when the task genuinely decomposes into independent parallel threads. Tightly serial tasks → single agent with strong context engineering.

EXPERIMENT 6 — single vs multi, same eval set

METRIC	SINGLE	MULTI
Report quality		
Citation correctness		
Coverage		
Source quality		
Wall-clock latency		
Model tokens		
Search requests		
Cost per successful report		

Note the last row's wording: **cost per successful task**, not token price. That is the economic metric that matters.

10.4 Subagent orchestration — four patterns, ordered by control

#	PATTERN	TOOLS	MAIN AGENT ROLE	MODEL REQUIREMENT
1	**Inline tool**	`call_agent`	caller	any tool-using model
2	**Fan-out**	`spawn_agent`, `wait_agent`	dispatcher — must reason about *when* to wait	capable
3	**Agent pool**	`spawn`, `send`, `wait`, `list`, `kill`	coordinator tracking multi-agent state	frontier handles 2–4
4	**Teams**	+ cross-agent `send_message`	supervisor	frontier-class for *every* agent

> `RULE` — Start at pattern 1 and stay there until an eval proves you need more. Most things that feel like they need multi-agent work fine as a well-prompted inline tool call.
>
> `TRAP` — **Do not let subagents spawn their own subagents.** Enforce it in the orchestration layer, *not* in the prompt. This is the runaway-cost failure mode.

10.5 Evaluating a research agent

No unit tests exist for this. Combine:

```
GROUNDEDNESS      every claim traceable to a retrieved source
COVERAGE          a defined list of key facts a good answer must contain
SOURCE QUALITY    authoritative, not merely first-retrieved
EXACT MATCH       for objectively-answerable sub-questions
LLM RUBRIC        synthesis coherence — RECALIBRATE FREQUENTLY against
                  your own judgement; research quality drifts and
                  experts disagree
```

Benchmark to study: **BrowseComp** (arXiv 2504.12516) — easy to verify, hard to solve.

Read

- Anthropic — *How we built our multi-agent research system* · https://www.anthropic.com/engineering/multi-agent-research-system
- Phil Schmid — *How Agents Manage Other Agents* · https://www.philschmid.de/subagent-patterns-2026
- Phil Schmid — *Deep Research with the Gemini API* · https://www.philschmid.de/deep-research-update
- LangChain Deep Agents · https://www.langchain.com/deep-agents
- DeepResearch Bench · arXiv 2506.11763
- Your saved videos · https://www.youtube.com/watch?v=cI2WTKzxgEE · https://www.youtube.com/watch?v=ARRD9itTMgw

`EXIT`

Your research agent produces a report where **every claim has a verified citation**, plus a cost-per-report number and a groundedness score.

Phase 11 — MCP and A2A

Time	~20 hours
Framework	ADK

> `RULE` — Build your own `ToolRegistry` **first**, then adapt it to MCP. Start with "let's make everything MCP" and you learn a protocol while skipping the thing the protocol abstracts.

11.1 MCP

```
MCP Client  ◄──────►  MCP Server

  tools  ·  resources  ·  prompts
  capability discovery  ·  schemas
  authentication  ·  authorization
```

Pay particular attention to **tool discovery** and **permissions** — those are the parts that matter for harness design. Current tool guidance emphasises validation, access controls, confirmation on sensitive operations, timeouts, and auditability.

Build: an MCP server exposing `search_documents`, `save_note`, `list_artifacts`, `get_project_metadata`. Then make your agent an MCP client. Then run the progressive-disclosure experiment again at protocol scale:

```
100 available MCP tools
        ▼
tool discovery / search
        ▼
   5 relevant tools
        ▼
       LLM
```

> `TRAP` — Blindly connecting MCP servers and exposing all their tools. This is the single most common MCP mistake in the wild, and it creates serious context bloat.

11.2 A2A — after MCP

```
MCP:   Agent  ◄──────►  Tools / Resources
A2A:   Agent  ◄──────►  Agent
```

Concepts: agent cards · tasks · messages · parts · artifacts. ADK 2.x has native A2A support, so this is cheap for you to try.

Build one tiny experiment — the main research agent delegating to a specialist agent over A2A:

```
Research Agent
     ├── MCP ──► search server
     ├── MCP ──► document server
     └── A2A ──► scientific-paper specialist
```

You do not need A2A throughout the application. The point is understanding the protocol boundary and when it is the right seam.

Read

- MCP specification · https://modelcontextprotocol.io/specification

- Phil Schmid — *How to correctly use MCP servers* · https://www.philschmid.de/use-mcp-servers
- Anthropic — *Code execution with MCP* · https://www.anthropic.com/engineering/code-execution-with-mcp
- A2A key concepts · https://a2a-protocol.org/latest/topics/key-concepts/

EXIT

A working MCP server and client of your own, plus a measured comparison of all-tools vs. discovered-tools at 100-tool scale.

Phase 12 — The serving layer

Time	~20 hours
Framework	ADK
Note	Compressible if you are not shipping to real users — but skipping it means you built an agent, not a product

These are the parts nobody blogs about and every real system has.

```
▸ STREAMING OVER SSE
  Budget TTFT and inter-token latency separately.

▸ RESUMABLE STREAMS
  Each streaming client holds an open connection. If it drops and
  reconnects it may land on a backend with no memory of the session.
  SSE supports resumption but YOUR BACKEND HAS TO IMPLEMENT IT.
  ► Fix: keep partial output in Redis, not in-process, so any
    instance can serve a reconnecting client.

▸ INTERRUPTION AND CANCELLATION
  Mid-generation, with correct persistence of the partial turn.

▸ BACKGROUND / LONG-RUNNING TASKS
  A research run takes minutes. The HTTP request cannot hold it.

▸ MULTI-TIER CACHING
  exact-match → semantic → prefix (KV) caching.
  Each with its own hit-rate metric. Cached responses return in
  single-digit ms vs hundreds for a fresh call.

▸ MODEL ROUTING BY COMPLEXITY
  cheap model      → simple conversational turns
  reasoning model  → hard multi-step tasks
  small structured → memory extraction
  ► Evaluate the quality/cost frontier. Do not assume.

▸ SESSION PERSISTENCE, TITLES, MESSAGE EDITING, BRANCH/REGENERATE
  State-model problems, not AI problems — and the reason ChatGPT
  feels like a product.
```

Read

- Redis — *Streaming LLM Responses* · https://redis.io/blog/streaming-llm-responses/
- Sebastian Raschka — *Controlling Reasoning Effort in LLMs* · https://magazine.sebastianraschka.com/p/controlling-reasoning-effort-in-llms

EXIT

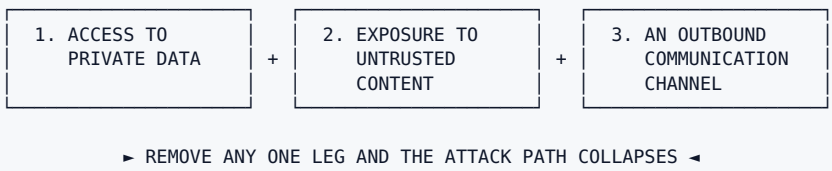
Kill the browser tab mid-research-run, reopen it, and the stream resumes with no lost output.

Phase 13 — Security and permission architecture

Time	~18 hours
Framework	ADK
Why it matters	This is where a toy agent becomes production engineering — and it is the phase that matters most for Project 2

13.1 The threat model — the lethal trifecta

An agent is exposed when it simultaneously has:



That is the whole model, and it is more useful than any list of attack names.

> `RULE` — 2026 assessments find the trifecta present in the overwhelming majority of production agents, and adaptive attacks bypass state-of-the-art prompt-injection classifiers a large fraction of the time. **Detection is not a defense.**

13.2 Classify every tool

LEVEL	DESCRIPTION	EXAMPLES
0	pure / read-only	`calculator`, `search`, `read_document`
1	reversible write	`save_note`, `create_draft`
2	consequential write	`send_email`, `create_calendar_event`, `modify_database`
3	destructive / high-risk	`delete_resource`, `execute_shell`, `transfer_money`, `deploy`

13.3 The approval gate

> `RULE` — **Do not ask the model to police itself.** The policy lives outside the model, in code, and is unit-testable.

13.4 Layer the defenses

```
   7  PRIVILEGE SEPARATION      ◄── the only structurally robust layer,
                                    because it does not depend on detection
   6  Monitoring
   5  Human approval on irreversible actions
   4  Sandboxing (network egress, filesystem scope, resource isolation)
   3  Tool-level scoping and allowlists
   2  OUTPUT GUARDRAILS         ◄── catches exfiltration on the way out;
                                    the layer that saves you when input
                                    filtering fails
   1  Input guardrails (cheap classifier on every request)
```

The two-tier pattern is the cost-effective shape: a cheap first-pass check on all traffic, escalating only the 10–15% it cannot resolve to an expensive LLM judge.

For your web/research agent specifically: **indirect prompt injection via fetched pages**, untrusted tool output, secret isolation, scope-limited credentials, output sanitization.

Read

- Simon Willison — *The lethal trifecta for AI agents*
- Meta AI — *Agents Rule of Two: A Practical Approach to AI Agent Security*
- Anthropic — *Beyond permission prompts* (sandboxing) · https://www.anthropic.com/engineering/claude-code-sandboxing
- Anthropic — *How we contain Claude across products* · https://www.anthropic.com/engineering/how-we-contain-claude
- OWASP Top 10 for LLM Applications

`EXIT`

A prompt-injection eval suite that your agent passes, and a permission policy that is enforced in code and covered by unit tests.

Phase 14 — Reliability and durable execution

Time	~18 hours
Framework	ADK

14.1 Budgets, not `while True`

```
RunBudget(
    max_steps        = ...,
    max_model_tokens = ...,
    max_tool_calls   = ...,
    max_searches     = ...,
    max_wall_time    = ...,
    max_cost         = ...,
)
```

14.2 Implement

```
  timeouts                    cancellation
  retries + exponential       durable checkpoints
    backoff                   queueing + backpressure
  idempotency keys            concurrency limits
  rate limits                 circuit breakers
  partial failure handling    model fallback
  tool fallback               cache invalidation
  resumable runs
```

14.3 The target behaviour

```
Run crashes after research step 19

✗  NOT →  restart research from zero

✓  BUT →  reload  plan · sources · notes · artifacts · run state
          and continue from step 19
```

This is exactly why Phase 10 wrote research state to files instead of keeping it in the conversation. The artifact design pays off here.

`EXIT`

Kill the process mid-research-run. Restart. It resumes and completes. Prove it with a `checkpoint_resume` eval.

Phase 15 — Cost, routing, and the improvement loop

Time	~20 hours
Framework	ADK

15.1 The loop that matters more than any prompt

> **`RULE`** — The most important loop in this project is **not**: `prompt → looks bad → edit prompt → looks better`

```
Observe a real production failure
        ▼
Inspect the trace
        ▼
Classify the failure
        ▼
WRITE AN EVALUATION CASE          ◄── this is the step people skip
        ▼
Change ONE system component
        ▼
Measure against the full suite
        ▼
Keep or revert
        ▼
Deploy → observe → (repeat)
```

Your eval dataset should grow primarily **from real failures**, not from imagination. Put them in `evals/failures/`.

15.2 The Swiss-cheese picture

METHOD	WHEN IT FIRES
Automated evals	every commit, every model upgrade — first line of defense
Production monitoring	post-launch drift, unanticipated failure
A/B testing	validating significant changes once you have traffic
User feedback	surfaces what you did not anticipate; sparse and severity-skewed
Manual transcript review	weekly habit; builds intuition nothing else gives you
Systematic human studies	calibrating your LLM judges

15.3 Two things to actively watch

```
▸ EVAL SATURATION
  An eval at 100% tracks regressions but gives ZERO improvement signal.
  SWE-bench Verified went from ~30% to >80% within about a year; large
  capability gains now show up as small score deltas.
  ► When a new model looks "unimpressive," suspect your eval before
    you suspect the model.

▸ INFRASTRUCTURE NOISE
  Resource allocation and enforcement methodology alone inject
  meaningful noise into agentic eval results.
  ► Do not conclude a 3-point change is real without repeated trials.
```

15.4 The payoff metric — the argument you will use at work

> **Model migration.** With a real eval suite, adopting a new model is a couple of days. Without one, it is weeks of manual vibes-testing. That is the compounding return on Phase 4.

`EXPERIMENT 7` — one strong model vs routed models

Measure accepted outcome per dollar, and latency.

`EXIT`

A dashboard showing cost per successful task, and at least three eval cases that originated from real failures.

Phase 16 — Framework comparison labs · `R1.5`

Time	~15 hours
Framework	read-only

Now — and only now, with a working system and an eval suite — read other frameworks. You will read them as "*here's how someone else solved the problem I just solved*" rather than as magic.

```
LangGraph            explicit state machines, checkpointing,
                     time-travel debugging, durable execution
                     ► the graph model ADK 2.0 moved toward

OpenAI Agents SDK    a deliberately small runtime.
                     Read it END TO END — it is short.
                     Primitives: agents, handoffs, guardrails, sessions

PydanticAI           clean Pythonic harness composition, typed

Claude Agent SDK     the Claude Code harness.
                     ► THIS IS PROJECT 2'S REFERENCE IMPLEMENTATION.
                       Read the source now.
```

For each, answer in your learning log: **what does this framework make easy that mine makes hard, and what did it decide that I decided differently?**

`EXIT`

Four written comparisons, each naming one idea you are going to steal for Phase 17.

Phase 17 — Remove ADK, build your own runtime · `R2` · CAPSTONE

Time	~35 hours
Framework	none

Rebuild in this order

```
1.  Agent loop + streaming            ←── you already have this from R0
2.  Session store and event model     ←── this is what ADK's Session/Event was
3.  Your own Runner, with callbacks/hooks at each lifecycle point
4.  Context assembly as an explicit, testable pure function
5.  Compaction and context editing BY HAND
      → then swap in the provider-native version and compare token counts
6.  Sub-agent spawning with real context isolation
7.  Your own OTel instrumentation emitting gen_ai.* spans
```

Target structure

```
agent_runtime/
├── models/          base.py  gemini.py  openai.py  anthropic.py
├── runtime/         loop.py  run.py  events.py  budgets.py
├── context/         builder.py  budget.py  compaction.py  provenance.py
├── tools/           base.py  registry.py  selection.py  executor.py  permissions.py
├── memory/          models.py  reader.py  writer.py  consolidation.py  scoring.py
├── retrieval/       retriever.py  hybrid.py  agentic.py  reranker.py
├── research/        planner.py  orchestrator.py  workers.py  citations.py
├── protocols/       mcp/  a2a/
├── security/        policy.py  approvals.py  sandbox.py
├── observability/   tracing.py  metrics.py  cost.py
└── evals/           datasets/  graders/  runners/
```

`EXPERIMENT 10` — the capstone

> **Run the same eval suite against R1 (ADK) and R2 (yours).**
>
> If R2 scores worse, find out **exactly which piece of the framework you under-built.** That diff is the most valuable single thing you will learn in Project 1 — it tells you precisely what the framework was doing for you that you never noticed.

`EXIT`

Your runtime passes the full eval suite, and you can explain every line of `loop.py` without hesitation.

Phase 18 — Advanced harness experiments · `R3`

Time	20+ hours, ongoing
Framework	yours

Only after the eval system is solid.

18.1 Programmatic tool calling

```
INSTEAD OF
   LLM → tool → LLM → tool → LLM → tool → LLM
   (every intermediate result lands in context)

TRY, for data-heavy tasks
   LLM
    ▼
   generate a small orchestration program
    ▼
   program calls several tools
    ▼
   filters / transforms intermediate data
    ▼
   condensed result
    ▼
   LLM
```

Anthropic documents this specifically as a way to avoid filling context with large intermediate tool results and to orchestrate dependent operations efficiently. Directly relevant to Project 2.

18.2 Evaluator–optimizer loops

```
Generator ⟶ candidate ⟶ Evaluator ⟶ feedback ⟶ Generator ⟶ ...
```

Useful when quality matters much more than latency. **`EXPERIMENT 9:`** generator-only vs generator + verifier on citation-heavy research tasks. Do not add reflection loops everywhere by default — measure whether they improve *accepted outcomes*.

18.3 Skills and progressive disclosure

```
skills/
├── scientific_research/SKILL.md
├── financial_analysis/SKILL.md
└── web_research/SKILL.md
```

The agent sees concise descriptions first and loads detailed instructions only when needed. Another form of context engineering.

18.4 Self-improvement — the disciplined version

> **`TRAP`** — Interpreting "self-improving" as *let the model rewrite its prompt whenever it wants.*

```
Production traces
      ▼
failure clustering
      ▼
candidate cause
      ▼
NEW EVAL CASE
      ▼
prompt / tool / context / code modification
      ▼
offline experiments
      ▼
regression suite
      ▼
HUMAN REVIEW
      ▼
deployment
```

18.5 Reactive loop vs explicit planning

`EXPERIMENT 8:` evaluate both on long-horizon tasks. Planning helps more as horizon grows; it costs latency on short tasks.

PART III — MEASUREMENT

19. The eval tree you should eventually have

```
evals/
├── core_chat/
│
├── tool_selection/
│   ├── should_call/
│   ├── should_not_call/          ◄── as important as should_call
│   ├── argument_correctness/
│   ├── retries/
│   └── termination/
│
├── context/
│   ├── long_context/
│   ├── compaction/
│   ├── stale_context/
│   └── tool_result_clearing/
│
├── memory/
│   ├── recall/
│   ├── update/
│   ├── forget/
│   ├── supersession/
│   ├── temporal/
│   └── isolation/                ◄── correctness property, not a nicety
│
├── retrieval/
│   ├── recall/
│   ├── reranking/
│   └── grounding/
│
├── research/
│   ├── coverage/
│   ├── citation_correctness/
│   ├── citation_completeness/
│   ├── source_quality/
│   └── contradiction/
│
├── security/
│   ├── prompt_injection/
│   ├── exfiltration/
│   ├── tool_permissions/
│   └── tenant_isolation/
│
├── reliability/
│   ├── transient_failure/
│   ├── cancellation/
│   ├── checkpoint_resume/
│   └── budget_enforcement/
│
├── failures/                     ◄── grown from real production failures
└── regression/                   ◄── gates CI, sits near 100%
```

20. Metrics to track from the beginning

```
┌─ QUALITY ──────────────────────┬─ EFFICIENCY ──────────────────────┐
│  task success                  │  input tokens                     │
│  pass@1                        │  output tokens                    │
│  pass^k          ◄── honest    │  cached tokens                    │
│  groundedness                  │  context size                     │
│  citation correctness          │  tool calls                       │
│  citation completeness         │  search calls                     │
│  memory recall / precision     │  model calls                      │
│  retrieval recall              │  agent steps                      │
│  tool-selection accuracy       │                                   │
│  argument correctness          │                                   │
│  security pass rate            │                                   │
├─ LATENCY ──────────────────────┼─ ECONOMICS ───────────────────────┤
│  TTFT                          │  cost per turn                    │
│  time to last token            │  cost per run                     │
│  tool latency                  │  COST PER SUCCESSFUL TASK  ◄──    │
│  retrieval latency             │  cost by model                    │
│  memory latency                │  cost by tool                     │
│  total run latency             │                                   │
│  P50 / P95                     │                                   │
├─ RELIABILITY ──────────────────┴───────────────────────────────────┤
│  retry rate · tool failure rate · model failure rate               │
│  cancellation rate · resume success rate · budget-exhaustion       │
└────────────────────────────────────────────────────────────────────┘
```

RULE — The headline metric is **cost per successful task**, not token price. A cheaper model that fails twice as often is more expensive.

21. The ten experiments

These teach more than any tutorial. Each gets an entry in `docs/experiments/`.

#	EXPERIMENT	PHASE	MEASURES
1	Raw loop vs ADK	5	code, traces, behaviour, failure handling
2	30 tools vs top-5 retrieval	6	input tokens, selection accuracy, latency, success
3	No memory vs 3 memory architectures	8	recall, precision, update/forget, cost
4	Vector vs BM25 vs hybrid vs agentic search	9	same corpus, same tasks
5	Long context vs compressed context	7	quality plotted against context tokens
6	Single vs multi-agent research	10	coverage, groundedness, latency, tokens, cost
7	One strong model vs routed models	15	accepted outcome per dollar, latency
8	Reactive loop vs explicit planning	18	long-horizon task success
9	Generator vs generator + verifier	18	citation-heavy research quality
10	**ADK runtime vs your custom runtime**	**17**	**the shared eval suite — the capstone**

PART IV — EXECUTION

22. Timeline

	PART	PHASE	HRS	CUMULATIVE @10h/wk	
	FOUNDATIONS	Part I (read + interfaces)	5	wk 1	
		0 · Setup	5	wk 1	
	R0	1 · Bare loop	15	wk 3	
		2 · Product shell	18	wk 5	
		3 · Observability	12	wk 6	
▶		4 · EVALUATION	30	wk 9	◄ the gate
	R1	5 · ADK migration	15	wk 10	
		6 · Tool engineering	18	wk 12	
		7 · Context engineering	22	wk 15	
▶		8 · MEMORY	35	wk 18	◄ the core
		9 · Retrieval	22	wk 21	
		10 · Research (single→multi)	30	wk 24	
		11 · MCP + A2A	20	wk 26	
		12 · Serving layer	20	wk 28	
		13 · Security	18	wk 29	
		14 · Reliability	18	wk 31	
		15 · Cost / routing / loop	20	wk 33	
	R1.5	16 · Framework labs	15	wk 35	
▶	R2	17 · CUSTOM RUNTIME	35	wk 38	◄ capstone
	R3	18 · Advanced harness	20+	wk 40+	
		TOTAL	~253	~9 months	

Realistic expectation: 8–10 months at 10 hrs/week if you genuinely perform the experiments and write the evals. You can finish a superficial version much faster. Do not optimise for that.

Compressible: Phases 2, 12, 14 if you are not shipping to real users. **Not compressible:** 4, 8, 17.

23. Weekly rhythm

At roughly 10 hours/week:

```
2 hrs   primary-source reading
5 hrs   implementation
2 hrs   evals / trace review / experiments
1 hr    learning notes + ADRs
```

> **TRAP** — 8 hours of courses and 2 hours of coding. **Invert that ratio.**

Friday checkpoint — 30 minutes, every week

1. What did I build this week?
2. What did the evals say?
3. What did I read, and did it change anything?
4. What is the top failure mode right now?
5. What goes in next week?

This is what keeps the reading list from growing faster than the code.

24. Documentation to keep while learning

```
docs/
├── learning-log.md              weekly, informal, honest
├── architecture-decisions/      ADR-001 … ADR-0NN
├── experiments/                 one file per experiment, template in §Phase 0
├── failure-taxonomy.md          from Phase 4 open/axial coding
└── benchmarks/
    ├── memory-comparison.md
    ├── retrieval-comparison.md
    └── single-vs-multi-agent.md
```

25. Portfolio artifacts to publish

Each is a blog post or a repo README section. Together they are a far stronger signal than "I did an agents course."

```
 1.  The agent loop, from scratch, explained
 2.  Run/event model and why messages[] is not enough
 3.  Trace-driven debugging of an agent
 4.  Your eval methodology + failure taxonomy
 5.  MEMORY BENCHMARK      ADK vs file-based vs hybrid   ◄ strongest
 6.  RETRIEVAL BENCHMARK   BM25 vs vector vs hybrid vs agentic
 7.  Research agent with verified citations + cost per report
 8.  Single vs multi-agent, with real metrics
 9.  Security architecture: deterministic policy, approvals, sandbox,
     injection evals
10.  ADK VS CUSTOM RUNTIME on the same eval suite   ◄ the capstone
```

26. Anti-patterns — the specific ways this project dies

```
✗  DON'T learn five frameworks simultaneously.
   Concepts transfer; framework APIs don't. One framework deeply,
   then your own runtime, then read the others as comparison.

✗  DON'T start with multi-agent.
   One excellent agent first. Multi-agent adds coordination cost,
   latency, token spend and debugging difficulty — and largely wins
   by spending more tokens.

✗  DON'T call your vector database "memory."
   It's a storage component. Memory is the policy for what gets
   persisted, updated, forgotten and exposed.

✗  DON'T start RL before evaluation.
   Without evals you have no reward signal and no way to know if
   anything improved.

✗  DON'T expose every tool to every request.
   Progressive disclosure is a core harness pattern now, not an
   optimisation.

✗  DON'T let LLM-as-judge be your only evaluator.
   Deterministic and outcome graders first.

✗  DON'T measure only answer quality.
   Quality, reliability (pass^k), latency, cost per successful task,
   tool efficiency, citation quality, memory correctness, safety.

✗  DON'T blindly increase reasoning tokens or context length.
   Both are resources, not improvements. Benchmark them.

✗  DON'T trust a benchmark number without its harness and token cost.
   See Appendix C.

✗  DON'T let the Udemy courses set your pace.
   Scaffolding for weeks 1–3, then drop them. Course material lags the
   field by 6–12 months; several numbers in Phases 7–10 post-date the
   typical refresh cycle.

✗  DON'T optimise before you can measure.
   This is the general form of every rule above.
```

PART V — REFERENCE

27. Reading, mapped to phase

Never read ahead. Attach each resource to the feature you are implementing.

PHASE	PRIMARY READING
1	Ball *How to Build an Agent* · Anthropic *Building effective agents* · Claude Agent SDK intro
3	Phoenix docs + OpenInference spec · OTel GenAI conventions · Hamel evals FAQ (tooling)
4	Anthropic *Demystifying evals* · **Hamel & Shreya evals FAQ, in full** · DeepEval docs · awesome-evals · your 4 eval videos
6	Anthropic *Writing effective tools* · *Advanced tool use* · *Code execution with MCP* · *Agent Skills* · *think tool* · Schmid *Agent Skills tips*
7	Anthropic *Effective context engineering* · Claude cookbook context-engineering · *Effective harnesses for long-running agents* · *Harness design*
8	Anthropic memory tool docs · ADK memory · Schmid *Memory in Agents* · MemGPT / Mem0 / LongMemEval papers · DeepLearning.AI memory courses · your memory video
9	Anthropic *Contextual Retrieval* · *Keyword Search Is All You Need* (arXiv 2602.23368) · Search-R1 (arXiv 2503.09516)
10	Anthropic *Multi-agent research system* · Schmid *Subagent patterns 2026* · BrowseComp · DeepResearch Bench
11	MCP spec · Schmid *Use MCP servers* · A2A key concepts
13	Willison *Lethal trifecta* · Meta *Agents Rule of Two* · Anthropic sandboxing + containment · OWASP LLM Top 10
12	Redis streaming · Raschka *Controlling reasoning effort*
15	Anthropic *Infrastructure noise* · production-monitoring material
16–17	LangGraph source · OpenAI Agents SDK source · Claude Agent SDK source · PydanticAI
parallel	RLHF Book · Raschka RL state-of · Schmid *Kimi/Cursor/Chroma RL*

28. Verdict on your existing resource list

```
┌─ KEEP AND PRIORITISE ──────────────────────────────────────┐
│                                                            │
│  Anthropic engineering blog                                │
│    Highest signal density for this project, by a wide margin.  │
│    https://www.anthropic.com/engineering                   │
│                                                            │
│  philschmid.de                                             │
│    Closest thing to a weekly agent-engineering journal.    │
│    He is at Google DeepMind — his ADK/Gemini coverage is   │
│    first-hand. Read the whole 2026 index; it's short.      │
│                                                            │
│  Sebastian Raschka                                         │
│    Best for the model layer. *Components of A Coding Agent*    │
│    is your bridge into Project 2.                          │
└────────────────────────────────────────────────────────────┘

┌─ ADD — MISSING FROM YOUR LIST ─────────────────────────────┐
│                                                            │
│  hamel.dev evals FAQ            ◄── the most important gap.    │
│    https://hamel.dev/blog/posts/evals-faq/                 │
│    Your eval quality will live or die on this.             │
│                                                            │
│  Shankar & Husain — *Evals for AI Engineers* (O'Reilly)    │
│  awesome-evals — github.com/benchflow-ai/awesome-evals     │
│  Arize Phoenix docs + OpenInference spec                   │
│    You named Phoenix as a goal — go to the primary source.     │
└────────────────────────────────────────────────────────────┘

┌─ DEMOTE ───────────────────────────────────────────────────┐
│  DeepLearning.AI courses    breadth, low depth — orientation   │
│                             only, not building             │
│  Udemy Complete Agentic AI   parallel breadth. Use it to ask   │
│                             "how does this framework implement  │
│                             a concept I already understand?"   │
│  Udemy LLM Engineering       low priority — you know this layer│
│  Udemy Gen/Agentic in Prod   save for Phases 12–15         │
└────────────────────────────────────────────────────────────┘

┌─ DROP ─────────────────────────────────────────────────────┐
│  OpenAI AgentKit / Agent Builder tutorials                 │
│    Product shuts down 30 Nov 2026. See §2.5.               │
└────────────────────────────────────────────────────────────┘
```

29. Optional parallel track — training and agentic RL

> **RULE** — Do this **in parallel** with Phases 8–15, never before. Agentic RL needs a meaningful environment, action space, trajectories, rewards and evaluation. Your project gives you all five. Without them, RL study is abstract.

```
SFT on verified trajectories
        ▼
tool-use training
        ▼
process / outcome reward
        ▼
RL over multi-step trajectories
```

- Nathan Lambert — **RLHF Book** · https://rlhfbook.com/
- RLHF playlist · https://www.youtube.com/watch?v=jQPiH-KB4B0
- RL playlist · https://www.youtube.com/watch?v=NFo9v_yKQXA
- Sebastian Raschka — *The State of RL for LLM Reasoning* (GRPO, RLVR) · https://magazine.sebastianraschka.com/p/the-state-of-llm-reasoning-model-training
- Sebastian Raschka — *LLM Research Papers: The 2026 List* — highest signal-per-minute reading list in the field
- **Phil Schmid — *How Kimi, Cursor, and Chroma Train Agentic Models with RL*** · https://www.philschmid.de/kimi-composer-context — **read this one even if you skip the rest**; it is the bridge between harness design and RL training
- Survey — *The Landscape of Agentic Reinforcement Learning for LLMs* · arXiv 2509.02547

- Z.ai GLM-4.5 technical report · https://z.ai/blog/glm-4.5
- HuggingFace RL blog tag · https://huggingface.co/blog?tag=rl

30. Definition of done for Project 1

You are ready for the coding-agent project when you can **answer and demonstrate** all of these.

```
┌─ RUNTIME ──────────────────────────────────────────────────┐
│  [ ] How does the agent loop work?                         │
│  [ ] What is a run? What is an event?                      │
│  [ ] How are stop reasons handled?                         │
│  [ ] How are cancellation and retry handled?               │
│  [ ] How are long-running tasks resumed?                   │
└────────────────────────────────────────────────────────────┘
┌─ CONTEXT ──────────────────────────────────────────────────┐
│  [ ] What is in the context, and WHY is each item there?   │
│  [ ] How do you budget tokens?                             │
│  [ ] When do you compact? What gets removed?               │
│  [ ] How does caching change prompt structure?             │
└────────────────────────────────────────────────────────────┘
┌─ TOOLS ────────────────────────────────────────────────────┐
│  [ ] How are tools described? How are they selected?       │
│  [ ] How are errors exposed?                               │
│  [ ] How are side effects classified?                      │
│  [ ] How are permissions enforced — and WHERE?             │
└────────────────────────────────────────────────────────────┘
┌─ MEMORY ───────────────────────────────────────────────────┐
│  [ ] What should be remembered? What should NOT be?        │
│  [ ] How are updates and supersession handled?             │
│  [ ] How does forgetting work?                             │
│  [ ] What is the provenance of a memory?                   │
│  [ ] How is retrieval evaluated?                           │
└────────────────────────────────────────────────────────────┘
┌─ RETRIEVAL ────────────────────────────────────────────────┐
│  [ ] When keyword, vector, hybrid, or agentic search?      │
│  [ ] How do you measure retrieval? How do you verify grounding? │
└────────────────────────────────────────────────────────────┘
┌─ RESEARCH ─────────────────────────────────────────────────┐
│  [ ] How does planning work? How do you decide when to stop? │
│  [ ] How are sources evaluated?                            │
│  [ ] How are claims mapped to evidence?                    │
│  [ ] When do subagents actually help?                      │
└────────────────────────────────────────────────────────────┘
┌─ EVALS ────────────────────────────────────────────────────┐
│  [ ] Task / trial / grader / trajectory / outcome — define each │
│  [ ] How do capability and regression evals differ?        │
│  [ ] What do pass@k and pass^k mean, and which do you report? │
│  [ ] How do real failures become tests?                    │
└────────────────────────────────────────────────────────────┘
┌─ OBSERVABILITY ────────────────────────────────────────────┐
│  [ ] Can you reconstruct a failure from a trace alone?     │
│  [ ] Do you know token/cost/latency BY COMPONENT?          │
│  [ ] Can you compare model/prompt/runtime versions?        │
└────────────────────────────────────────────────────────────┘
┌─ SECURITY ─────────────────────────────────────────────────┐
│  [ ] Can the model directly execute high-risk actions?     │
│  [ ] Where is the policy enforced?                         │
│  [ ] What happens with untrusted web content?              │
│  [ ] What secrets can tools access?                        │
│  [ ] How is cross-user state isolated?                     │
└────────────────────────────────────────────────────────────┘
┌─ PRODUCTION ───────────────────────────────────────────────┐
│  [ ] What are the run budgets?                             │
│  [ ] How do retries work? Are tool operations idempotent?  │
│  [ ] Can work resume after failure?                        │
│  [ ] What is the cost per successful task?                 │
└────────────────────────────────────────────────────────────┘
┌─ FRAMEWORK INDEPENDENCE ───────────────────────────────────┐
│  [ ] Can ADK be removed?                                   │
│  [ ] Can you implement the loop yourself?                  │
│  [ ] Can you explain what the framework was doing for you? │
│  [ ] Can you swap Gemini for Claude without rewriting the agent? │
└────────────────────────────────────────────────────────────┘
```

If all boxes are checked, Project 1 has served its purpose.

31. Final study order

If you ever lose track, return to this list.

```
01  Raw model SDK                  13  Multi-agent research
02  Manual tool loop               14  MCP
03  Basic chat product             15  A2A
04  Run / event model              16  Serving + resumability
05  Tracing                        17  Security + permissions
06  EVALUATION SYSTEM              18  Durable execution
07  ADK mapping                    19  Cost / caching / routing
08  Tool engineering               20  Production feedback loop
09  Context engineering            21  Framework comparison labs
10  MEMORY                         22  FULL CUSTOM RUNTIME REBUILD
11  Retrieval                      23  Advanced harness experiments
12  Single-agent research          24  Coding-agent project
```

The ordering is intentional. In particular:

```
┌──────────────────────────────────────────────────────┐
│   EVALS BEFORE MEMORY                                │
│   SINGLE AGENT BEFORE MULTI-AGENT                    │
│   TOOL REGISTRY BEFORE MCP                           │
│   RUNTIME UNDERSTANDING BEFORE FRAMEWORK DEPENDENCE  │
│   MEASUREMENT BEFORE OPTIMIZATION                    │
└──────────────────────────────────────────────────────┘
```

APPENDICES

Appendix A — Transition into Project 2: the coding agent

Do not start Project 2 until the eval and runtime ideas above are comfortable. Nearly everything transfers.

```
PROJECT 1                          PROJECT 2
Chat / Research Agent              Coding Agent
──────────────────────────────────────────────────────

web search                  →      filesystem + grep/glob
documents                   →      repository map, editor, patches
research notes              →      shell, compiler, tests, git, sandbox

ToolRegistry                →      filesystem / shell / git tools
ContextManager              →      repository context selection
Memory                      →      repo / session / procedural memory
Artifacts                   →      patches, plans, test logs
Permissions                 →      shell / filesystem / network policy
Run budget                  →      coding-task budget
Research subagents          →      coding subagents
Eval harness                →      SWE-bench / terminal tasks
Tracing                     →      command / model / edit trajectories
Checkpoints                 →      long-running coding sessions
```

That is why this sequence beats cloning Claude Code directly. Project 1 teaches the generic harness. Project 2 applies it to an unusually demanding environment — where the tools are destructive, the feedback is real (tests pass or fail), and the permission model actually matters.

Round structure for Project 2

```
Round 1   use a batteries-included coding framework
Round 2   build a minimal coding loop yourself
Round 3   build a serious coding harness inspired by
          Claude Code · Codex · OpenCode · pi · Cursor
```

Warm-up reading — start after Phase 13

- Sebastian Raschka — *Components of A Coding Agent* — your bridge into Project 2
- Anthropic — *Claude Code: Best practices for agentic coding*
- Anthropic — *How we built Claude Code auto mode: a safer way to skip permissions*
- Anthropic — *Beyond permission prompts* (sandboxing)
- Anthropic — *Building a C compiler with a team of parallel Claudes*
- Anthropic — *Scaling Managed Agents: Decoupling the brain from the hands*
- **The Claude Agent SDK source itself** — the reference harness
- OpenAI cookbook — *Build a coding agent* · *Using PLANS.md for multi-hour problem solving* · *Build Code Review with the Codex SDK*
- Cursor research · https://cursor.com/blog/topic/research
- Terminal-Bench 2.0 (via Harbor) · SWE-bench Verified
- Phil Schmid — *Evaluating Agents Beyond the First Prompt*

Appendix B — Primary reference index

Anthropic

```
Engineering hub          https://www.anthropic.com/engineering
Building Effective Agents      /building-effective-agents
Demystifying Evals             /demystifying-evals-for-ai-agents
Effective Context Engineering  /effective-context-engineering-for-ai-agents
Writing Tools for Agents       /writing-tools-for-agents
Advanced Tool Use              /advanced-tool-use
Code Execution with MCP        /code-execution-with-mcp
Agent Skills                   /equipping-agents-for-the-real-world-...
Multi-Agent Research           /multi-agent-research-system
Contextual Retrieval           /contextual-retrieval
Harness Design                 /harness-design-long-running-apps
Long-Running Harnesses         /effective-harnesses-for-long-running-agents
Claude Code Sandboxing         /claude-code-sandboxing
How We Contain Claude          /how-we-contain-claude
```

Google

```
ADK                    https://adk.dev/get-started/python/
ADK 2.0 migration      https://adk.dev/2.0/
A2A protocol           https://a2a-protocol.org/
DeepMind Research      https://deepmind.google/research/
```

OpenAI

```
Developers             https://developers.openai.com/
Cookbook               https://developers.openai.com/cookbook
Agents SDK             https://openai.github.io/openai-agents-python/
Deprecations           https://developers.openai.com/api/docs/deprecations
```

Evaluation and observability

```
Hamel Evals FAQ        https://hamel.dev/blog/posts/evals-faq/
DeepEval               https://deepeval.com/docs/introduction
Arize Phoenix          https://arize.com/docs/phoenix/
awesome-evals          https://github.com/benchflow-ai/awesome-evals
```

Frameworks (to read)

```
LangGraph              https://docs.langchain.com/oss/python/langgraph/
PydanticAI             https://ai.pydantic.dev/
Claude Agent SDK       https://platform.claude.com/docs/
MCP                    https://modelcontextprotocol.io/
```

People and research

```
Phil Schmid            https://www.philschmid.de/
Sebastian Raschka      https://magazine.sebastianraschka.com/
Hugging Face blog      https://huggingface.co/blog
RLHF Book              https://rlhfbook.com/
DeepLearning.AI        https://www.deeplearning.ai/courses
arXiv                  https://arxiv.org/
```

Appendix C — Claim audit

> Both AI-generated roadmaps this document was merged from assert specific numbers with equal confidence. Here is the tiering. **Apply the same tiering to anything you add.**

`VERIFIED` — checked against primary or near-primary sources, 25 Aug 2026

▸ ADK Python 2.0 GA 19 May 2026; ADK Go 2.0 GA 30 June 2026.
  Workflow Runtime = graph-based execution engine.
  Breaking changes to agent API, event model, session schema.
  2.0 sessions readable by 1.28+ but not older 1.x. PyPI current 2.5.0.
  → google/adk-python, adk.dev, PyPI

▸ OpenAI Agent Builder deprecation announced 3 June 2026.
  Evals read-only 31 Oct 2026. Both shut down 30 Nov 2026.
  Agents SDK is the Agent Builder migration path;
  PROMPTFOO is the Evals migration path.
  → OpenAI deprecations page

▸ Claude Agent SDK exposes the same loop / tools / context management /
  permissions / subagents as Claude Code.
  Separate monthly Agent SDK credit from 15 June 2026:
  $20 Pro / $100 Max 5x / $200 Max 20x.

▸ Claude Code dropped RAG + local vector DB for agentic search.
  Stated reasons: simplicity, security, privacy, staleness, reliability.
  → Boris Cherny, primary quote widely reproduced

▸ Mem0 reported at BOTH 49.0% and 94.4% on LongMemEval across
  different harnesses/years. Zep 63.8%. Independent academic tables
  show yet another ordering.
  → multiple sources incl. arXiv 2604.07877 comparison table

`VENDOR` — directionally useful, not decision-grade

▸ Anthropic multi-agent research: ~90.2% over single-agent Opus 4,
  ~15× tokens, token usage explaining ~80% of variance.
  Internal eval, their workload.

▸ Memory tool + context editing: ~84% token reduction, ~39% performance
  improvement on a 100-turn web-search eval. Self-reported.

▸ Mem0's own LoCoMo 92.5 / LongMemEval 94.4 at ~7k tokens/query.

▸ Prompt-caching cost and latency reduction ranges.

► TREAT EVERY NUMBER IN THIS TIER AS A HYPOTHESIS TO TEST ON YOUR
  OWN WORKLOAD. That is what Phases 4, 8 and 9 are for.

`UNVERIFIED` — check before relying on

▸ Several arXiv IDs quoted in the source roadmaps, particularly the
  26xx-series. The Amazon Science "Keyword Search Is All You Need"
  paper (arXiv 2602.23368) and Search-R1 (arXiv 2503.09516) appear in
  multiple independent write-ups; others appear only once.

► OPEN THE PAPER BEFORE YOU CITE IT.

Appendix D — The habit this project should teach

If you finish with nothing else, finish with this.

That habit is worth more than any single agent framework, and it is the thing that will still be true after every framework in this document has been rewritten twice.

End of Project 1 roadmap. Project 2 — the coding agent — deserves its own document at this depth.